Week 6: Energy-based models

Jin-Guo Liu, Xi-Wei Pan

Abstract: Generative modeling learns a probability distribution from examples and supplies a way to draw new samples. An energy-based model expresses that distribution through an energy function, as Week 3's Ising model did. We first inspect the datasets and their observed sample frequencies. We then learn an energy for handwritten digits, generate new ones, and complete partly observed digits by inference. Next we set Ising couplings so that chosen patterns become energy minima, and measure how many patterns this memory holds. Finally, a modern form of the memory retrieves digits in one step, and that step is the attention operation of language models. These experiments teach us to formulate an energy model, distinguish recovering an input image (reconstruction) from drawing a new one (generation), and test a model against a known physical result. Each Part's Goal states the result you will judge.

Generative modeling

Learning a distribution from examples. Suppose we have a collection of images and want to draw new ones with similar objects, shapes and variety. Each image is a sample from an unknown probability distribution over pixel values.

The generative-modeling task. Let x denote one data item, such as an image or a spin configuration. Let p_{data} be its unknown probability distribution. Given N examples drawn from this distribution, write the dataset as $\mathcal{D} = \{x^{(1)}, \ldots, x^{(N)}\}$. Generative modeling fits a model distribution p_θ with parameters θ and provides a procedure for sampling from it. The two requirements are

$$p_\theta \approx p_{\text{data}}, \quad x_{\text{new}} \sim p_\theta. \tag{1}$$

Here $x^{(i)}$ is example i, x_{new} is a new sample, and $\sim$ means "is sampled from." Training uses $\mathcal{D}$ to choose θ; generation draws new samples with those parameters fixed. The figure denotes the fresh random input by $z \sim p_z$, where p_z is a chosen noise law, such as a standard Gaussian: independent normal coordinates of mean zero and variance one. A sampler can also draw fresh noise during its steps.

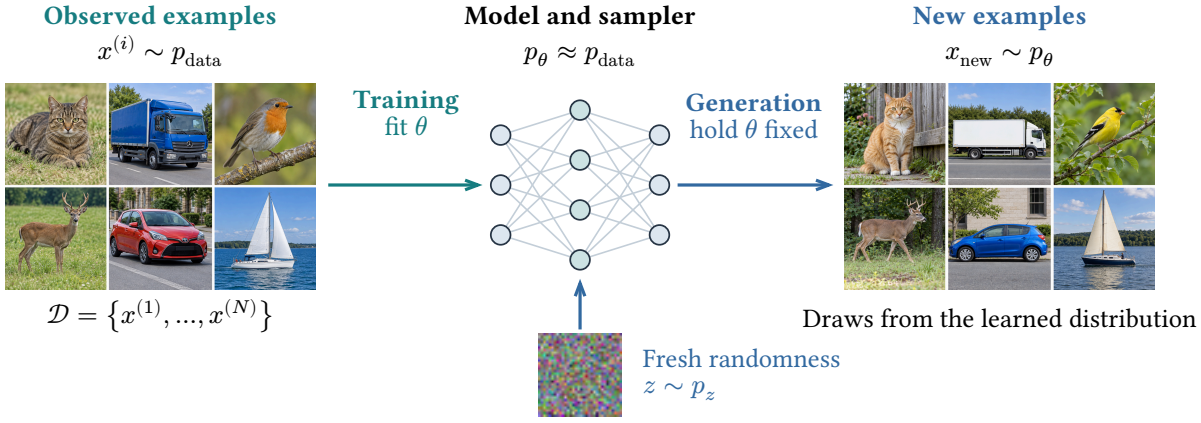


Figure 1: Training fits θ from observed examples; generation holds θ fixed and uses fresh randomness. The photo panels are high-resolution, AI-generated illustrations of example distributions. They are not experimental outputs; Part 2 shows the measured model samples. The network is schematic.

Agreement between the distributions means reproducing the frequencies and variation of data properties. We judge that agreement through sample statistics, such as image features or energy fluctuations. A model can provide a sampling procedure even when evaluating $p_\theta(x)$ is difficult.

From Monte Carlo to a learned model. In Week 3, an energy function and a temperature specified the distribution we sampled. Here the training examples specify the target. We must learn a model of that distribution before we can draw new samples from it.

The three building blocks of neural-network training.

- *Model and parameters.* A neural network is a function built from layers of adjustable calculations. Its architecture specifies how those layers connect. Its parameters, collectively called θ, are the numerical weights and biases that training adjusts.

- *Loss.* A loss measures disagreement between a model and its training data. Its definition depends on the model and the learning task.
- *Optimization method.* An optimizer updates the parameters to reduce the loss. Week 5's reverse-mode automatic differentiation computes such gradients. The loss gradient contains the derivatives with respect to the parameters. Gradient descent moves opposite that gradient; the learning rate controls the update size. Repeating updates on *batches*, small groups of examples, is training.

Resources:

- Lei Wang, *Generative Models for Physicists*. Sections 1.1 and 1.2.1 connect probability modeling to statistical mechanics, then explain how to infer a spin model from examples. Use this overview with Part 2.
- Geoffrey Hinton, *A Practical Guide to Training Restricted Boltzmann Machines*. Sections 2–3 explain how to fit image probabilities; section 5 explains why reconstruction error can mislead. Section 14 extends training by continuing sampling chains between updates. Use it with Part 2.
- Liu, Li and Wang, *Marburg restricted Boltzmann machine hands-on notebook*. Compare training and reconstruction with Part 2's spin model. Part 2 adapts this sequence to JAX, Week 5's array and automatic-differentiation library, and adds fresh generation.

Part 1: Understand the datasets

Goal: image galleries with digit or object labels and plots of the fraction in each class (class frequencies) that identify what one sample represents and how the distributions compare between data used for fitting (the training split) and data reserved for evaluation (the test split). This is the training/test split.

① DO

Work in `week6/` inside your existing exercise repository. CPU is the baseline; GPU acceleration is optional. Download the data and send the build prompt, then read UNDERSTAND while your agent works.

```
Work in week6/ using uv, with CPU as the default. Save figures in evidence/ and raw
runs with settings in artifacts/. Keep downloads and raw runs out of git. Commit after
each task; ask about uncertainties instead of guessing.
```

A label identifies an image's digit or object class. MNIST contains handwritten digits in grayscale, one brightness value per pixel; CIFAR-10 contains color photographs with red, green and blue (RGB) channels. Download the images and labels from the CVDF MNIST mirror and the binary archive from the CIFAR-10 authors. Keep these files for the later exercises. The commands append their download URLs to `README.md`.

```
mkdir -p inputs/mnist inputs/cifar10
mnist_url=https://storage.googleapis.com/cvdf-datasets/mnist
for split in train t10k; do
  for kind in images-idx3-ubyte labels-idx1-ubyte; do
    name="$split-$kind.gz"
    curl -fL "$mnist_url/$name" -o "inputs/mnist/$name" \
      --write-out "%{url_effective}\n"
  done
done >> README.md
cifar_host=https://mindspore-website.obs.cn-north-4.myhuaweicloud.com
curl -fL "$cifar_host/notebook/datasets/cifar-10-binary.tar.gz" \
  -o inputs/cifar10/cifar-10-binary.tar.gz \
  --write-out "%{url_effective}\n" >> README.md
```

Image galleries display several images together. Preserving native pixels means showing the original image resolution without smoothing or inventing detail.

```
Build reusable visualization Python scripts for grayscale and RGB images with optional
labels: image galleries and class-frequency plots by training/test split. Preserve
native pixels.
```

2 UNDERSTAND

We want to establish what distribution the downloaded examples represent by examining their source, image representation, training/test splits and class frequencies.

Dataset downloads. Common places to find datasets include Hugging Face Datasets, Kaggle Datasets, and the UCI Machine Learning Repository. Follow a dataset's original project page when available. The dataset descriptions, often READMEs or dataset cards, explain who collected it, what it contains, how it is split and its usage terms. Dataset provenance records the source and version so the examples you model can be identified. The direct downloads above need no hub account.

Grayscale and RGB images. A grayscale image stores one brightness value per pixel; an RGB image stores three values for red, green and blue. MNIST represents a handwritten digit as a 28×28 grayscale array. CIFAR-10 represents a photograph as a 32×32 array with three color channels. Both downloads store integer pixel values from 0 to 255.

Labels and image galleries. A label identifies the digit or object class of an image. Each dataset has ten classes. A gallery displays several examples together, with labels making their classes visible. It shows the variety within a class and the differences between classes. An equal number of displayed images per class helps compare their appearances; it does not show their frequencies in the dataset.

The image and label must stay paired when reading the downloads. CIFAR's binary records store a label followed by the red, green and blue pixel planes. MNIST stores images and labels in separate compressed files in matching order. Preserving native pixels means displaying these arrays without smoothing or inventing detail. Enlarging a gallery makes pixels easier to see but adds no image information.

Training/test split. The training split supplies examples for fitting; the test split supplies held-out examples for evaluation. Later we fit a model to MNIST training images and inspect reconstructions of test images. CIFAR-10 supplies real color photographs; Week 7 samples a model already trained on them.

Class frequencies. A class frequency is the fraction of examples carrying a particular label. It describes how common that class is in the dataset. A finite dataset defines an empirical distribution that places probability $\frac{1}{N}$ on each of its N records; adding these weights over a class gives its frequency. For each split's plot, N is the number of records in that split. Write the label as y and its empirical probability as $\hat{p}$; for a class c, this fraction is

$$\hat{p}(y = c) = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}\left[y^{(n)} = c\right], \tag{2}$$

where $y^{(n)}$ is the label of record n, and the indicator is one when the label equals c and zero otherwise. This is a measured fraction of the dataset, not the unknown population probability. Our later generators produce images x without receiving a requested digit or object label y. Here labels help us describe the data.

KEY CONCEPTS

- **Provenance:** identifies the dataset's source and version; its description explains contents and usage.
- **Images and labels:** images are the samples we model; labels identify classes used to summarize them.
- **Native pixels:** MNIST is grayscale and CIFAR-10 is RGB; enlarging either adds no image detail.
- **Training/test split:** separates fitting from held-out evaluation; each split defines an empirical distribution.
- **Empirical distribution:** gives each record equal weight; class frequencies sum those weights by label.
- **Galleries and class frequencies:** examples reveal appearance; counts reveal how common each class is.

3 VERIFY

(1) **MNIST contains varied handwriting and unequal class frequencies.**

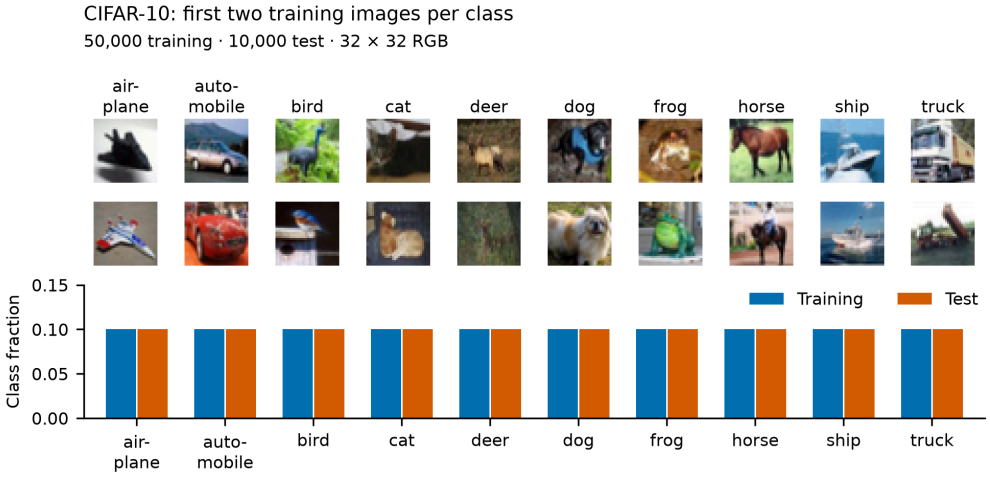
```
Visualize each dataset's empirical distribution through several labeled examples
per class and relative class frequencies in the training and test splits. Generate
one figure for each dataset. Explain the image representations.
```

Confirm 60000 training images and 10000 test images, preserving their original split assignments. Compare handwriting within each digit class. Identify the 28×28 grayscale representation and 0–255 pixel range. A balanced gallery helps compare appearances but hides unequal class frequencies; use every label in the frequency plot.

Compare the class fractions: digit 1 is most common and digit 5 least common in both splits. The reference shows downloaded images in stored order; its frequency plot counts every label.



(2) **CIFAR-10 balances classes at low image resolution.** Match labels to objects and inspect which details are lost at 32×32 pixels with three color channels. Confirm 50000 training images and 10000 test images in their original splits. Each class has 5000 training images and 1000 test images, so every plotted fraction is 0.1. Compare this balance with MNIST. Explain what the gallery tells you about image content and what the frequency plot tells you about the empirical distribution.



Part 2: Learn a generative spin model of handwritten digits

Goal: compare reconstructing an input digit, completing a partly observed digit, and generating a new one, and explain how training and the number of sampling steps affect the images.

① DO

Save this block as `week6/rbm.toml` in your exercise repository.

rbm.toml

```toml
[training]
images             = 60_000 # Use the full training split
epochs             = 20     # Passes through the training split
batch_size         = 100    # Images per parameter update
shuffle_each_epoch = true
optimizer          = "SGD"  # Plain gradient descent
learning_rate      = 0.1
momentum           = 0.0    # No momentum
weight_decay       = 0.0    # No weight penalty
```

```
precision          = "float32"
seed               = 6602   # Shared data-shuffle and Gibbs seed

[initialization]
weight_std           = 0.01 # Standard deviation of zero-mean normal weights
hidden_bias          = 0.0
visible_bias         = "logit(clipped training pixel frequency)"
pixel_frequency_clip = [0.01, 0.99] # Keep the visible biases finite
seed                 = 6601 # Shared parameter-initialization seed

[evaluation]
test_images       = 64 # First images in stored test order
# Keep Gibbs randomness fixed across epochs and models.
seed              = 6606
reconstruction_at = "initialization and each epoch"
surrogate         = "epoch mean of batch surrogates before updates"

[completion]
test_images = 64 # First images in stored test order
chains      = 8  # Per image and model
steps       = [1, 10, 100, 1000]
seed        = 6607
wrong_upper = "next evaluation image with a different label, wrapping within the 64
images"
```

```
1. (Model and sampling) Build a JAX restricted Boltzmann machine for MNIST with 128
hidden units. Binarize at half intensity. Implement energy, visible free energy and
binary block Gibbs sampling.
2. (Training) Train CD-1 and CD-10 using `rbm.toml`. Differentiate the free-energy
difference with stop-gradient on sampled configurations. Match initialization, data
order and evaluation randomness. Record settings, the CD surrogate and held-out one-
step reconstruction error (fraction of changed pixels).
```

(2) UNDERSTAND

In Week 3, a given energy function determined which spin configurations were probable. Here the configurations come from MNIST, and we learn an energy function that makes those examples probable. Once learned, that same model can generate new configurations by sampling.

Binarization and spin representation. Using the pixel range from Part 1, we binarize at half intensity: $v_i = 1$ for a pixel value at least 128, and $v_i = 0$ otherwise. The equivalent spin is $s_i = 2v_i - 1$. Each 28×28 image becomes a configuration of 784 spins, with ink $+1$ and background -1. The model learns these configurations without using the digit labels.

Restricted Boltzmann machine and energy. A restricted Boltzmann machine (RBM) represents an image with visible bits v and introduces unobserved hidden bits h to model relations between pixels. A hidden bit can favor a group of pixels turning on together, such as a stroke. "Restricted" means couplings connect the two layers only; there are none within either layer. Unlike a nearest-neighbour spin model, the image grid does not determine which pixels can interact through a hidden bit.

Let i index visible bits and j hidden bits. The bias vectors a, b favor individual bits being on, and the coupling W_{ij} controls the interaction between v_i and h_j. Together these parameters form θ. The energy and joint Boltzmann probability are

$$E_\theta(v,h) = -a^T v - b^T h - v^T W h, \quad p_\theta(v,h) = \frac{\exp(-E_\theta(v,h))}{Z_\theta}. \tag{3}$$

Here superscript T means transpose. The partition function Z_θ sums $\exp(-E_\theta(v,h))$ over every joint configuration. Temperature is absorbed into the parameters, so the energy is dimensionless. Lower energy gives higher joint probability.

In the diagram, $p(h|v)$ denotes the probability of hidden bits given fixed visible bits; $p(v|h)$ reverses these roles. The parameter subscript is suppressed.

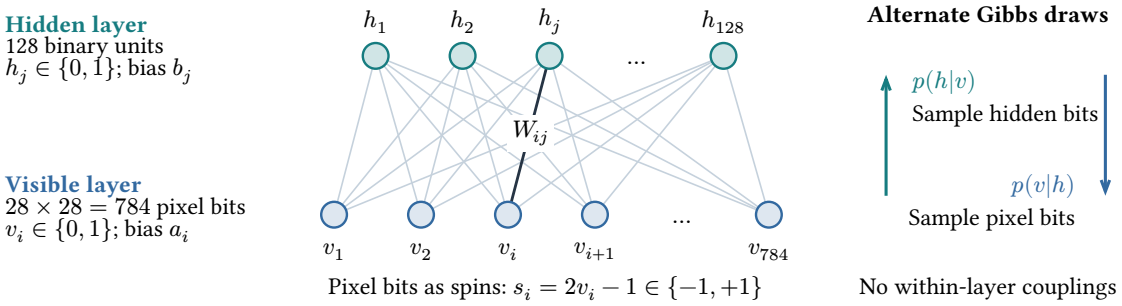


Figure 2: An RBM for MNIST. The reference uses 128 hidden units; only a few nodes and couplings are drawn. The arrows show alternating draws of one layer conditioned on the other, used for sampling below.

Visible free energy. The data contains images v, not hidden configurations h. To assign a probability to an image, sum over all its possible hidden states. The visible free energy performs this sum:

$$F_\theta(v) = -\log \sum_h \exp(-E_\theta(v,h)) = -a^T v - \sum_j \log\Big(1 + \exp\Big(b_j + (v^T W)_j\Big)\Big). \tag{4}$$

The sum factorizes because hidden bits are independent when the visible layer is fixed. Thus $p_\theta(v) = \frac{\exp(-F_\theta(v))}{Z_\theta}$: for fixed parameters, lower free energy means a more probable image. Computing $F_\theta(v)$ is easy, but normalizing over all 2^{784} visible images to obtain Z_θ is infeasible.

Training should increase the probability assigned to the observed images. Maximum-likelihood fitting expresses this as minimizing their average negative log probability, the negative log likelihood (NLL). For N training images $v^{(1)}, \ldots, v^{(N)}$, the loss is

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{n=1}^{N} \log p_\theta\big(v^{(n)}\big) = \mathbb{E}_{\text{data}}[F_\theta(v)] + \log Z_\theta. \tag{5}$$

Here $\mathbb{E}_{\text{data}}$ is the empirical average introduced in Part 1. The first term rewards low free energy for training images; the normalization term accounts for the probabilities of all the other images. This is why training cannot simply lower the free energy of every configuration.

Block Gibbs sampling. We can sample the model without computing Z_θ. Block Gibbs sampling alternates a draw of the entire hidden layer given v with a draw of the entire visible layer given h. Each pair of draws is one Gibbs step, also called a *sweep* because it redraws every bit once. The conditional probabilities follow from the RBM energy:

$$p_\theta(h_j = 1|v) = \sigma\left(b_j + \sum_i v_i W_{ij}\right), \tag{6}$$

$$p_\theta(v_i = 1|h) = \sigma\left(a_i + \sum_j W_{ij} h_j\right), \tag{7}$$

where $\sigma(u) = \frac{1}{1+\exp(-u)}$ is the logistic sigmoid, mapping a real input to a probability between zero and one. The bits within each draw are conditionally independent, so they can be sampled in parallel.

Repeated steps form a *Markov chain*: the next configuration depends only on the current one. Its stationary distribution is $p_\theta(v)$, meaning that a state drawn from this law still has the same law after another step. A short chain can retain its starting image; *mixing* describes how the chain loses that memory and explores the model distribution.

CD-1 and CD-10. The likelihood loss tells us what to optimize; its gradient tells us how to change the parameters. Write ∇_θ for the gradient with respect to θ. Differentiating the loss gives

$$\nabla_\theta \mathcal{L} = \mathbb{E}_{\text{data}}[\nabla_\theta F_\theta(v)] - \mathbb{E}_{p_\theta}[\nabla_\theta F_\theta(v)]. \tag{8}$$

The first expectation, a probability-weighted average, uses the training images. The second uses the model distribution and comes from differentiating the normalization term. Estimating it accurately requires well-mixed model samples, which can be expensive.

Contrastive divergence (CD-k) approximates that second expectation with short Gibbs chains. Start each chain at a training image v_0, take k Gibbs steps, and use its endpoint v_k as the model sample. CD-1 and CD-10 use one and ten steps. Increasing k changes the approximation and costs more sampling work. Matching initialization, data order and the other training settings lets us compare these choices.

For a coupling W_{ij}, the resulting update is proportional to

$$\mathbb{E}\big[v_{0,i}p_\theta(h_j = 1|v_0)\big] - \mathbb{E}\big[v_{k,i}p_\theta(h_j = 1|v_k)\big]. \tag{9}$$

These average visible-hidden products measure *correlations*. The data term is called the *positive phase*; the endpoint term is the *negative phase*. Their difference adjusts the couplings to make the model's correlations resemble those in the data. Short chains give an approximate likelihood gradient, not an exact one.

CD surrogate, automatic differentiation and stop-gradient. To compute this update in JAX, construct a scalar whose derivative is the CD gradient. A *batch* is a group of training images used for one update. Average the free-energy difference between each image and its chain endpoint over the batch to obtain the CD surrogate:

$$\tilde{\mathcal{L}}_{\mathrm{CD}} = \mathbb{E}_{\mathrm{batch}}[F_\theta(v_0) - F_\theta(\mathrm{sg}(v_k))]. \tag{10}$$

The stop-gradient operation sg holds the sampled endpoints fixed while automatic differentiation takes the derivative with respect to θ. JAX calls it `stop_gradient`. This reproduces the difference of free-energy gradients above; differentiating how the endpoints depend on θ would produce a different update.

The surrogate supplies the update, but its value is not the NLL: it replaces the normalization term with sampled free energies. Its history therefore need not follow the likelihood loss. Repeating updates across the training set completes one *epoch*, after which further epochs can continue fitting the model.

Training settings and initialization. The TOML file fixes the training budget and gradient-descent settings for the comparison. The weights start as small zero-mean normal draws; the hidden biases start at zero. For each pixel, let p be its frequency of being on in the training images. Clip p to $[0.01, 0.99]$ and initialize the visible bias to its *logit*, $\log\left(\frac{p}{1-p}\right)$. With zero couplings, this bias gives the pixel probability p; clipping keeps the bias finite for pixels that are always off or on. Seeds identify the random streams used for initialization, shuffled training and evaluation.

Held-out one-step reconstruction error. A held-out image belongs to the test split from Part 1 and is excluded from fitting. Reconstruction starts a Gibbs chain at that image and takes one step. Its error is the fraction of changed pixels, averaged over the held-out images, also called the *Hamming fraction*. Use the same held-out images and Gibbs randomness at initialization and after each epoch, so changes reflect the fitted parameters. Measure sampled binary pixels, as specified by the sampler, rather than thresholded probabilities. A low error can mean that the chain has barely moved from its input; it does not by itself show that the model can generate digits from scratch.

Fresh generation instead starts with random spin configurations, whose spins are independently -1 or $+1$ with equal probability. Keep the trained parameters fixed and run the same Gibbs sampler. Here image structure must emerge without a supplied digit. The sampling length controls how far this fixed model's chain evolves; k in CD-k controls the training approximation. A model trained with CD-1 can be sampled for 1000 steps without becoming a CD-1000 model. Longer sampling cannot repair a wrongly learned distribution.

Image completion as inference. Often only part of an image is observed. Split the visible bits into observed bits v_O and missing bits v_M. Image completion draws the missing bits from the model's conditional distribution

$$p_\theta(v_M \mid v_O) = \frac{p_\theta(v_O, v_M)}{\sum_{v'_M} p_\theta(v_O, v'_M)}. \tag{11}$$

Evaluating this ratio directly needs a sum over every configuration of the missing bits. Block Gibbs sampling avoids that sum. Draw the hidden layer given the full visible layer, draw the visible layer given the hidden layer, then clamp the observed bits: reset them to their observed values. Each draw leaves the conditional distribution of the hidden and missing bits, given the observed bits, unchanged. The missing bits of the clamped chain therefore have $p_\theta(v_M \mid v_O)$ as their stationary distribution. Answering a question about unobserved variables with the parameters fixed is called *inference*. Training fits θ; inference uses it.

Completion lies between the two earlier tasks. Reconstruction starts from the full image, and generation observes nothing. Here the upper half of a held-out digit is observed, and the lower half must be inferred. Several chains from the same observation can give different completions, because several digits can share an upper half.

Ink overlap on the missing half. Ink covers about 12% of the missing pixels of a held-out digit. A blank lower half therefore disagrees with only 12% of the missing pixels, so the fraction of wrong pixels favors blank completions. The ink overlap compares inked pixels only. Let A be the inked missing pixels of a completion and B those of the held-out digit. The overlap is $2|A \cap B|/(|A| + |B|)$: one for identical ink, zero for no shared ink and for a blank completion. A completion uses its observation when it overlaps the true lower half more than completions clamped to a wrong upper half, taken from another held-out digit.

KEY CONCEPTS

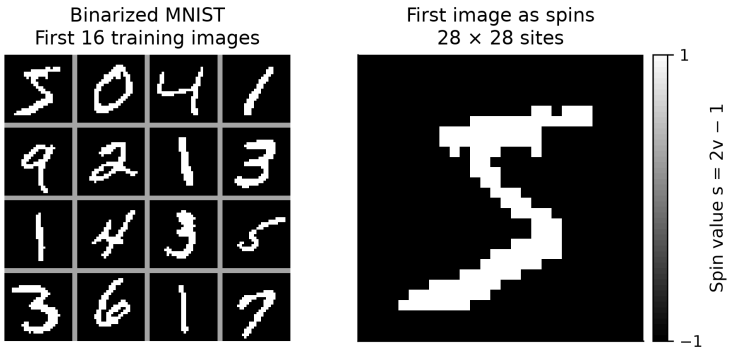
- **Spins and the RBM:** binarized pixels are visible spins; hidden bits model their relations through learned couplings.
- **Energy and visible free energy:** summing over hidden states turns the joint energy model into an image probability.
- **Free energy and NLL:** the training loss includes both the data's free energy and the model's normalization.
- **Gibbs sampling and CD:** alternating conditional draws supply the short chains used to approximate the likelihood gradient.
- **CD surrogate and stop-gradient:** differentiating the free-energy difference with sampled endpoints fixed gives the CD update.
- **Reconstruction and generation:** the same sampler starts from a held-out image or random spins; these assess different behavior.
- **Image completion:** clamping the observed bits turns the Gibbs sampler into a sampler of the conditional distribution of the missing bits; this is inference with fixed parameters.

3 VERIFY

(1) **The image is a configuration of visible spins.**

```
Visualize binarized MNIST images as spin configurations and diagram the RBM's
visible and hidden layers and their couplings.
```

Compare your binary images and network with the diagram in Figure 2: each image supplies 784 visible bits, and every learned coupling joins a visible bit to a hidden bit. There are no within-layer edges. Identify the pixels corresponding to spins $+1$ and -1. Reference images in this Part come from CPU JAX runs checked independently with NumPy using the TOML settings; individual stochastic trajectories can differ. The gallery shows the first 16 training images and one enlarged spin configuration.

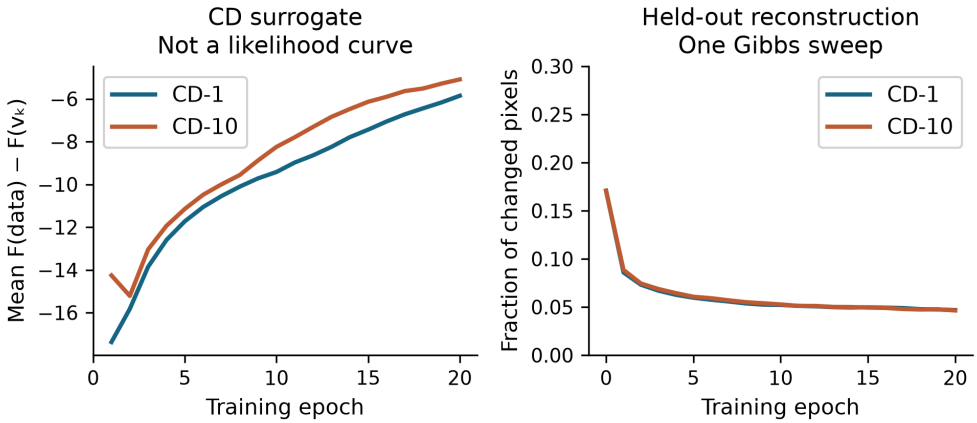


(2) **The CD surrogate and reconstruction error measure different things.**

```
Plot the CD surrogate and held-out one-step reconstruction error during training
for CD-1 and CD-10.
```

Check that CD-1 and CD-10 used the same initialization and training settings except for their chain length. Read the two curves separately: the surrogate need not decrease, whereas the held-out one-step reconstruction error should improve from initialization. The surrogate's sampled endpoints change with the model during training, so its value can rise while reconstruction improves. The plot labels the fraction of changed pixels as the Hamming fraction. Explain why neither curve alone establishes good fresh generation. Exact NLL also needs $\log Z_\theta$, which

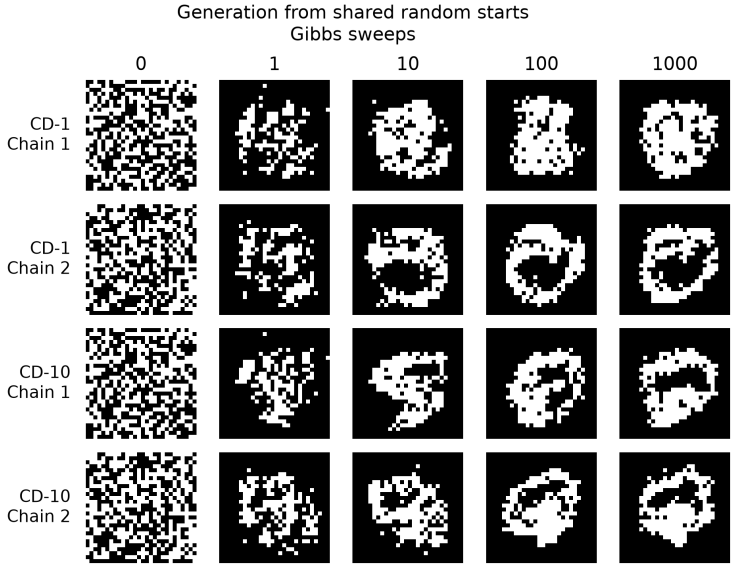
is infeasible to compute for this MNIST RBM. It changes during training, so omitting it does not give an NLL curve. These two diagnostics do not establish that NLL decreases. With 128 hidden units and 20 epochs on the reference CPU, training took about 18 seconds for CD-1 and 105 seconds for CD-10. Their one-step reconstruction errors fell from about 17% to 4.7% of pixels. Your error should fall to less than half of its value at initialization.



(3) **Generation develops structure from random spins.**

> Compare reconstruction from held-out digits with fresh generation from random spin configurations. Freeze both RBMs and use block Gibbs sampling from matched starts across models. Retain and visualize initial configurations and samples after 1,10,100,1000 steps to compare CD-1 and CD-10 as sampling length increases.

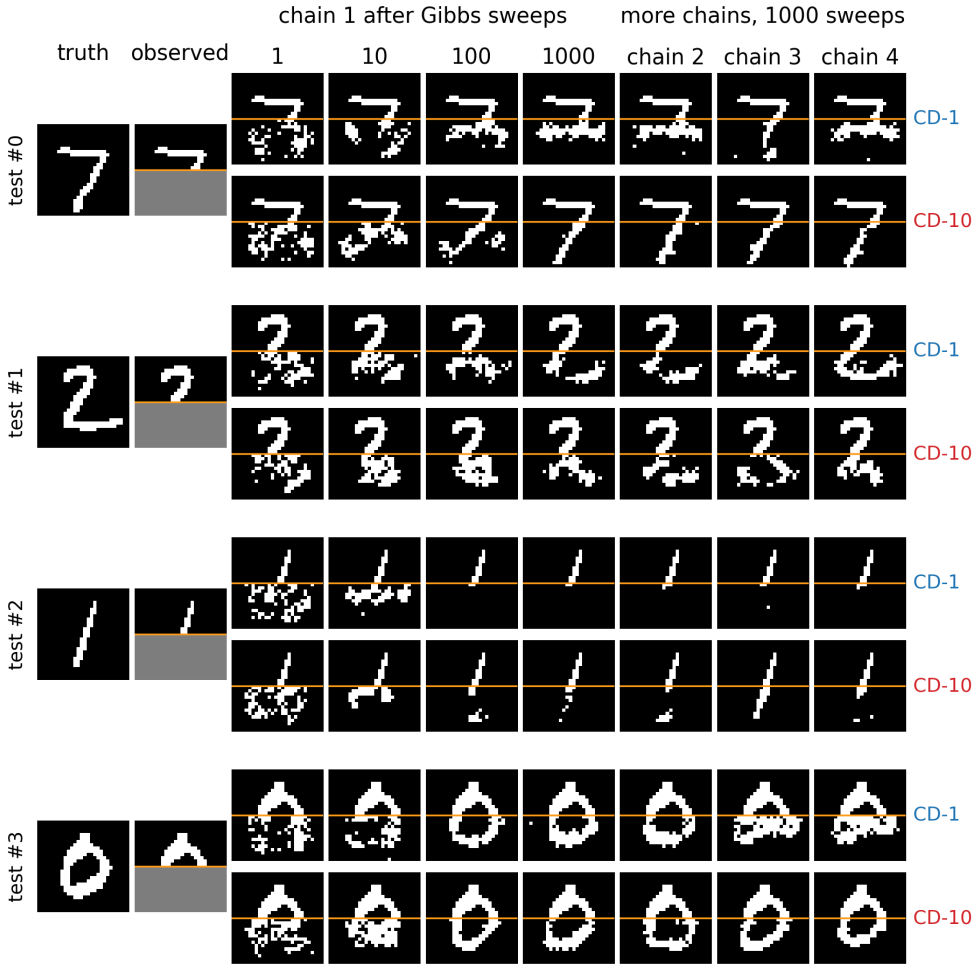
Confirm that both models start from the same random configurations, so differences reflect the fitted models rather than different starts. Inspect chains and galleries alongside the learning curves. Follow each chain through 1, 10, 100 and 1000 steps. Compare recognizable strokes, malformed digits and repeated shapes with the reconstructions. A recognizable one-step reconstruction can retain its starting digit; fresh generation must develop structure without one. Describe the differences you observe when changing training k and when changing generation length. Compare several chains before calling a pattern a trend; individual samples vary. The reference shows the first two chains at every step without selecting the best outputs. Some samples may remain poor: neither longer CD training chains nor longer generation chains guarantee a well-learned, well-mixed distribution. Structure means connected regions of ink with stroke-like edges, in place of the isolated pixels of the random start. The reference samples after 1000 steps are such blobs and rings, mostly not recognizable digits; a sample of that kind passes. A model this small, trained this briefly, rarely generates clean digits.



(4) **Clamped Gibbs sampling completes held-out digits.**

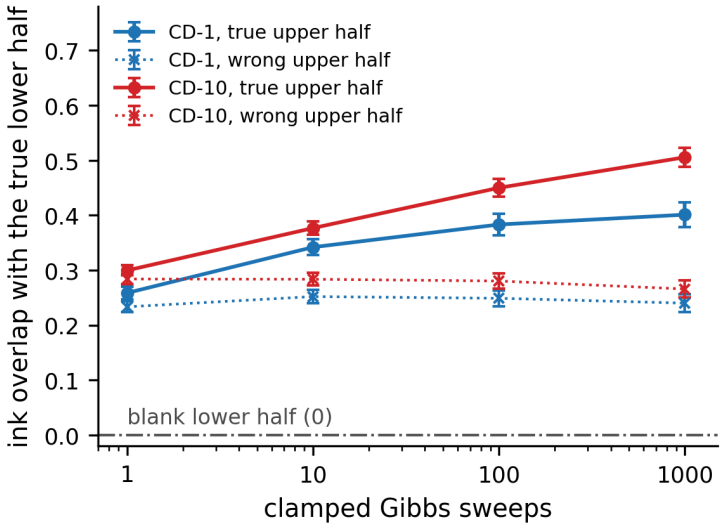
> Perform image completion on held-out digits whose lower half is missing. Clamp the observed upper half and resample the missing pixels with block Gibbs sampling from random starts. Use several chains per digit and both RBMs. Show completions after 1,10,100,1000 steps.

The observed upper half stays identical in every frame. After one step the lower half is speckle. After 1000 steps it holds strokes that continue the observed ones. Different chains can complete the same upper half differently: in the reference, the upper half of a 2 also yields 3-like completions. CD-1 often leaves the lower half of a 1 blank. The reference completes 64 held-out digits with 8 chains each; with the comparison below, it took about 5 minutes on a two-core CPU.



> Plot the ink overlap (2 |A ∩ B| / (|A| + |B|)) between completions and the true lower half against steps, for both RBMs. Repeat with each digit clamped to the wrong upper half, taken from another digit.

With the true upper half, the overlap grows with the number of steps. With a wrong upper half it stays flat, near 0.25, not zero: strokes of different digits share pixels near the image center. The reference reaches 0.51 ± 0.02 for CD-10 and 0.40 ± 0.02 for CD-1 after 1000 steps, against 0.27 and 0.24 for wrong upper halves; each $\pm$ is the standard error over the 64 digits. Your true-half overlap after 1000 steps should exceed your wrong-half overlap by at least 0.1 for both RBMs. At one step the two cases nearly agree: a short chain has not yet used the observation. CD-10 completes better than CD-1, although their one-step reconstruction errors agree in item 2.



Optional extension: estimating held-out NLL. Annealed importance sampling (AIS) estimates the partition function through intermediate distributions connecting a tractable reference model to the trained RBM. See Hinton's guide, §§5–6.

```
At a few saved training states, estimate held-out NLL with annealed importance
sampling and compare CD-1 with CD-10. Repeat independent estimates and increase the
sampling budget to assess stability. Plot estimated NLL alongside the existing
diagnostics.
```

Part 3: Store random patterns in a Hopfield network

Goal: show that Hebbian couplings make stored patterns stable states of an Ising energy, that the fraction of unstable bits follows the crosstalk prediction, and that recall collapses near the load $\alpha_c \approx 0.138$.

(1) DO

```
Build a Hopfield network in JAX for N spins s_i=±1. Store P patterns with independent
fair ±1 entries xi^mu with the Hebbian rule J_ij=(1/N) sum_mu xi_i^mu xi_j^mu and
J_ii=0. Recall at zero temperature with asynchronous updates s_i <- sign(h_i), where
h_i=sum_j J_ij s_j, in random order until a sweep flips no spin. Leave s_i unchanged
when h_i=0. Measure the overlap m=(1/N) sum_i xi_i s_i between a pattern and a state.
```

```
Record the fraction of unstable bits, those with xi_i h_i<0, over all stored patterns.
Start recall at the first 10 stored patterns of each set, or at all of them when P<10,
and record the final overlap. Sweep the load alpha=P/N from 0.05 to 0.25 in steps of
0.01 for N=100,200,400,800, with 20 independent pattern sets per point. Record seeds
and runtime.
```

(2) UNDERSTAND

Part 2 learned couplings by gradient descent so that an energy model generates digits. Here we set couplings in one step so that chosen configurations become low-energy states, and measure how many such states the energy can hold.

Hopfield network and patterns. A Hopfield network is an Ising model whose couplings store a list of configurations, the patterns $\xi^1, \ldots, \xi^P$, each a vector of N spins $\xi_i^\mu = \pm 1$. Started from a corrupted or partial version of a pattern, its dynamics returns the complete pattern. This retrieval by content is called *associative memory*. In this Part the patterns are random: every entry is $+1$ or -1 with equal probability, independently.

Hebbian rule and energy. The Hebbian rule sets each coupling to the average product of the two spins over the stored patterns,

$$J_{ij} = \frac{1}{N}\sum_{\mu=1}^{P} \xi_i^\mu \xi_j^\mu, \quad J_{ii} = 0. \tag{12}$$

Spins that agree in most patterns get a positive coupling. The energy is Week 3's Ising energy with these couplings in place of a lattice,

$$E(s) = -\frac{1}{2}\sum_{i,j} J_{ij} s_i s_j = -\frac{N}{2}\sum_{\mu=1}^{P} m_\mu(s)^2 + \frac{P}{2}. \tag{13}$$

The second form uses the overlap $m_\mu(s) = \frac{1}{N}\sum_i \xi_i^\mu s_i$, which is 1 when s equals pattern μ, -1 for its spin-reversed copy, and near 0 for an unrelated state. States with a large overlap with some pattern therefore have low energy.

Asynchronous update. The *local field* on spin i is $h_i = \sum_j J_{ij} s_j$. An asynchronous update visits one spin at a time and sets $s_i \leftarrow \mathrm{sign}(h_i)$, leaving the spin unchanged when $h_i = 0$. This is Week 3's Metropolis rule at zero temperature: a flip is accepted only when it lowers the energy. Each flip lowers E by $2|h_i|$, and E is bounded below, so recall stops after finitely many flips. It stops at a local minimum, which may or may not be close to a stored pattern.

Load and crosstalk. The load $\alpha = P/N$ counts patterns per spin. Start at pattern ν and split its local field into the pattern's own contribution and the *crosstalk* from the other patterns:

$$\xi_i^\nu h_i = \frac{N-1}{N} + \frac{1}{N}\sum_{\mu\neq\nu}\sum_{j\neq i} \xi_i^\nu \xi_i^\mu \xi_j^\mu \xi_j^\nu. \tag{14}$$

The crosstalk sums $(P-1)(N-1)$ random signs divided by N. It is close to a Gaussian with mean zero and variance $(P-1)(N-1)/N^2$, which approaches α for large N. Bit i is an unstable bit when $\xi_i^\nu h_i < 0$; its update moves the state away from the pattern. The predicted unstable fraction is

$$\Phi\left(-\sqrt{\frac{N-1}{P-1}}\right) \approx \Phi\left(-1/\sqrt{\alpha}\right), \tag{15}$$

where Φ is the standard normal cumulative distribution function. At $\alpha = 0.138$ the large-N form gives 0.36%. Finite N lowers it: at $N = 100$ and $\alpha = 0.10$ the first form is 40% below the second.

Storage capacity. Unstable bits flip, change the local fields of the other spins, and can trigger further flips. Amit, Gutfreund and Sompolinsky solved the large-N limit with the replica method of spin-glass theory. A recall state with overlap about 0.97 or higher exists for $\alpha < \alpha_c \approx 0.138$ and disappears above it; recall then ends far from the pattern. At finite N the drop is smooth, and it sharpens as N grows. See MacKay, *Information Theory, Inference, and Learning Algorithms*, chapter 42, for the capacity argument.

KEY CONCEPTS

- **Hopfield network and Hebbian rule:** couplings built from stored patterns turn them into low-energy Ising states.
- **Overlap and energy:** the energy decreases as the overlap with a stored pattern grows.
- **Asynchronous update:** zero-temperature single-spin updates lower the energy until recall stops at a local minimum.
- **Load and unstable bits:** crosstalk from other patterns grows with α and destabilizes a fraction $\Phi(-1/\sqrt{\alpha})$ of bits.
- **Capacity:** the recall state near a pattern survives only below $\alpha_c \approx 0.138$.

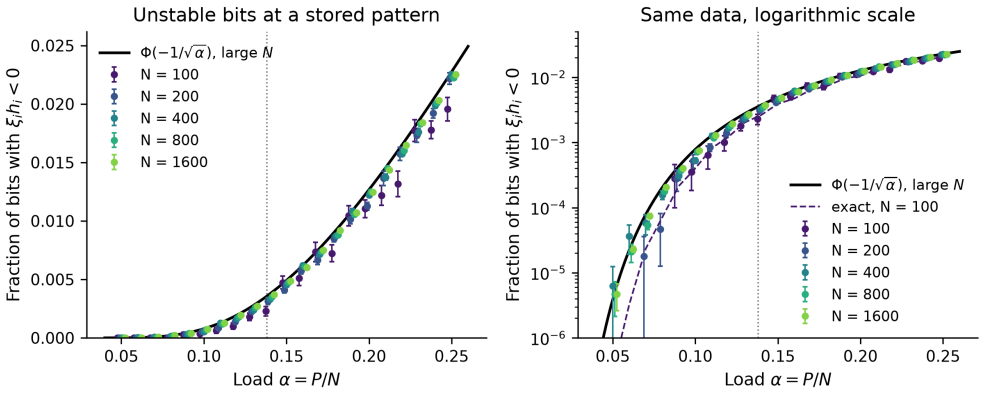
3 VERIFY

(1) **The unstable-bit fraction follows the crosstalk prediction.**

```
Plot the unstable-bit fraction against the load for each N, with standard errors
across pattern sets, beside the Gaussian crosstalk prediction Phi(-1/sqrt(alpha)).
```

The points approach the curve as N grows. At $N = 800$ and $\alpha = 0.20$ the reference measures 1.25×10^{-2} against 1.27×10^{-2}. The $N = 100$ points lie below the curve: at $\alpha = 0.10$ the reference measures 3.5×10^{-4} against 7.8×10^{-4}. The finite-N form predicts 4.6×10^{-4} there and accounts for most of the gap. A fraction near

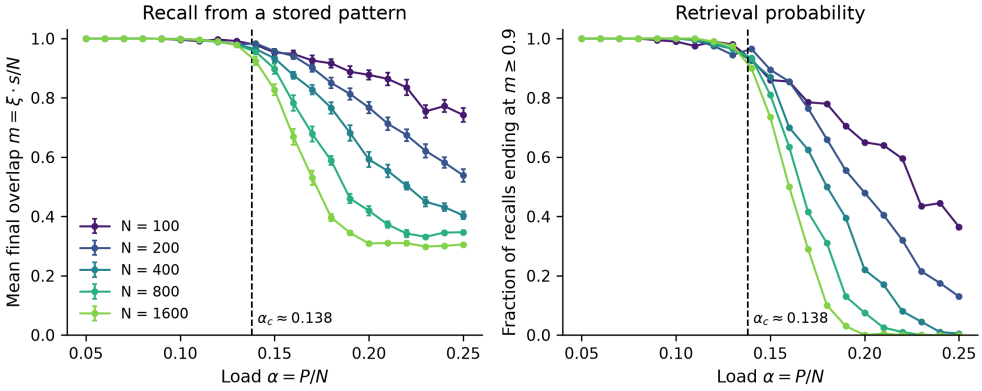
0.5 betrays couplings built from patterns other than the stored ones. A nonzero diagonal adds α to every $\xi_i h_i$ and pushes bits away from instability; the fraction then comes out about four times too small. The reference figure adds $N = 1600$ for comparison; it is not required.



(2) **Recall collapses near the storage capacity.**

> Plot the mean final overlap after recall, and the fraction of recalls ending at overlap at least 0.9, against the load. Draw one curve per N and mark alpha=0.138.

In the reference run, mean overlap stays above 0.988 up to α=0.12, indicating nearly complete recall. Beyond the mark the overlap drops, and the drop steepens as N grows. In the reference, the mean overlap falls below 0.9 at $\alpha = 0.186$, 0.170, 0.156 and 0.150 for $N = 100$, 200, 400 and 800, each ± 0.002 to 0.007. The crossing moves toward 0.138 as N grows. The reference also ran $N = 1600$: it crosses at 0.142, and above the drop its overlap settles near 0.30. The threshold 0.9 separates the recall state, with overlap about 0.97 or higher, from those low overlaps. Your crossings should decrease with N and lie within 0.02 of the reference values. The study up to $N = 800$ took about 7 minutes on a two-core CPU.



Part 4: Retrieve digits with a modern Hopfield network

Goal: show that the modern Hopfield update is one step of softmax attention, that the inverse temperature moves retrieval from an average of stored digits to one stored digit, and that modern retrieval succeeds at storage loads where classical recall of the same digits fails.

① DO

> Store the first P MNIST training images in stored order, binarized as spin patterns, the columns of X. Implement the modern Hopfield update x <- X softmax(beta X^T x) with inverse temperature beta. Compute the same update with JAX's softmax-attention function, using x as the query, the stored patterns as keys and values, and beta as the scale.

```
Query 100 stored digits, cycling through the stored ones, after replacing each lower
half with random spins. Retrieve with one modern update and with the classical
Hopfield recall on the same patterns. Record retrieval success for
P=3,10,30,100,300,1000 and beta=0.001,0.003,0.01,0.03,0.1,0.3,1. Success means the
sign of the result has overlap at least 0.95 with its target, and the target is the
nearest stored digit.
```

2 UNDERSTAND

Completion and retrieval. The queries here pose Part 2's completion task: the upper half of a digit is observed. The RBM completes digits it never saw, by sampling a learned distribution. A Hopfield network retrieves one of the digits it stored. Retrieval succeeds when the query identifies its stored digit.

Correlated patterns. Part 3's estimate of crosstalk assumed random patterns. Binarized digits are far from random: 87% of their pixels are background, and images of the same digit share strokes. Their pairwise overlaps are large, so the crosstalk is large and has a sign. In the reference, 12% of the bits of an uncorrupted stored digit are already unstable at $P = 100$. The shared background acts like one extra pattern that every image contains. Recall flows to a common state close to the pixelwise majority of the stored digits, whatever the query.

Modern Hopfield network and inverse temperature. A *modern Hopfield network* replaces the quadratic energy by a log-sum-exp over stored patterns. With the patterns as the columns of X and a real-valued state x, its energy is

$$E(x) = -\frac{1}{\beta}\log\sum_{\mu=1}^{P}\exp(\beta x^{\mu}\cdot x) + \frac{1}{2}x\cdot x + \text{const.} \tag{16}$$

The inverse temperature β controls how sharply the energy prefers the most similar pattern. The first term has the form of Part 2's visible free energy: it sums over a hidden choice of one pattern μ with weight $\exp(\beta x^{\mu}\cdot x)$.

Softmax and the update. The softmax of a vector u has components $\text{softmax}(u)_{\mu} = \exp(u_{\mu})/\sum_{\nu}\exp(u_{\nu})$. They are positive and sum to one. The modern Hopfield update

$$x \leftarrow X\,\text{softmax}(\beta X^{T}x) \tag{17}$$

replaces the state by a weighted average of stored patterns. Each weight grows exponentially with the pattern's similarity to the state. The update never raises E. For large β the weights concentrate on the most similar pattern, and one update retrieves it. For small β the weights are nearly uniform, and the update returns an average of many stored digits. Ramsauer and coauthors showed that well-separated patterns are retrieved in one update with exponentially small error, and that the number of storable random patterns grows exponentially with the number of pixels.

Attention. Softmax attention takes a query vector q, a matrix of *keys* K and a matrix of *values* V, one row per stored item, and returns

$$\text{attention}(q, K, V) = V^{T}\,\text{softmax}(\beta K q). \tag{18}$$

The query is compared with every key; the output averages the values with those weights. With $q = x$ and $K = V = X^{T}$, attention is exactly the modern Hopfield update. A transformer applies this step at every position of a sequence, with learned maps producing queries, keys and values; Week 8 builds one. See Ramsauer et al., *Hopfield Networks is All You Need* (blog) for the energy and the retrieval theorems.

Retrieval success. Take the sign of each component of the retrieved state. Retrieval success requires an overlap of at least 0.95 with the target, at most 19 wrong pixels of 784. Different stored digits are also similar: a typical digit has another stored digit at overlap 0.88. Success therefore also requires the target to be the nearest stored digit.

KEY CONCEPTS

- **Correlated patterns:** shared background and strokes make crosstalk large, so classical recall of digits fails at small P.
- **Modern Hopfield energy and inverse temperature:** a log-sum-exp over stored patterns, a free energy over a hidden choice of pattern; β sets its sharpness.
- **Softmax and the update:** the new state is a softmax-weighted average of stored patterns; large β retrieves one, small β averages many.

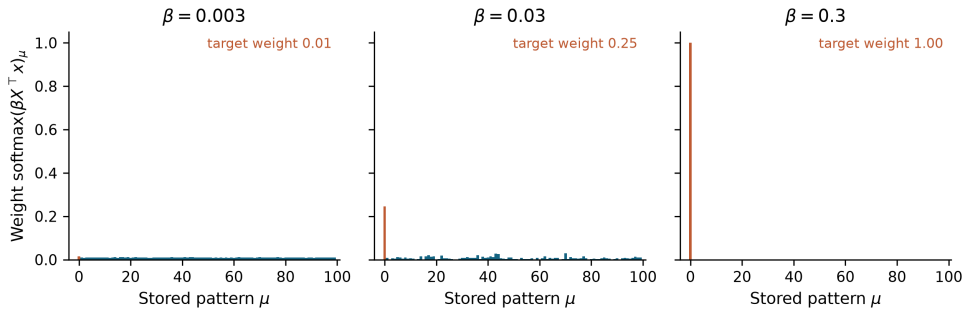
- **Attention:** a query compared with keys weights the values; with keys and values equal to the stored patterns this is the Hopfield update.

3 VERIFY

(1) **The modern update is one softmax-attention step.**

> Plot the softmax weights over stored patterns for one query at three beta values. Print the largest difference between the update and the attention output.

The largest difference is below 10^{-5}; the reference prints 6.0×10^{-7} over all queries, P and β, with 32-bit floating-point numbers. A scale other than β, such as the library's default $1/\sqrt{784}$, gives differences near 2. With 100 stored digits, the queried digit carries weight 0.01, 0.25 and 1.00 at $\beta = 0.003$, 0.03 and 0.3.

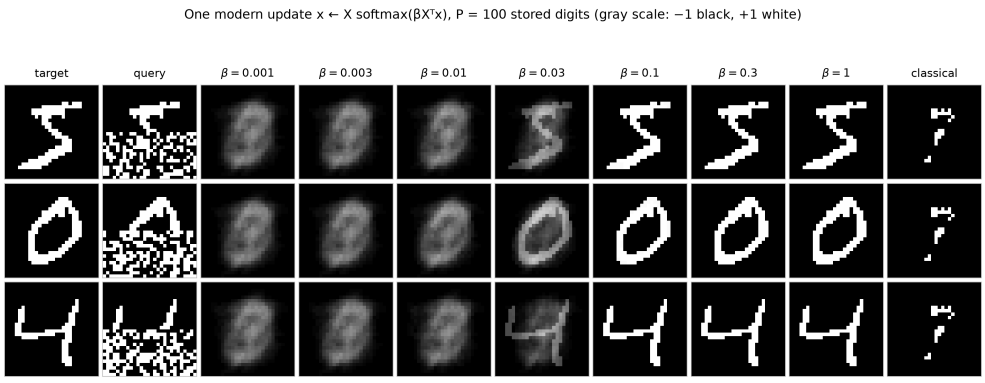


P = 100 stored digits; query 0 targets pattern 0 (orange). Max |modern update − jax.nn.dot_product_attention| over all queries, P and β: 6.0e-07

(2) **Inverse temperature separates averaging from retrieval.**

> Show several queries with their modern retrievals across beta, beside the classical recall of the same queries.

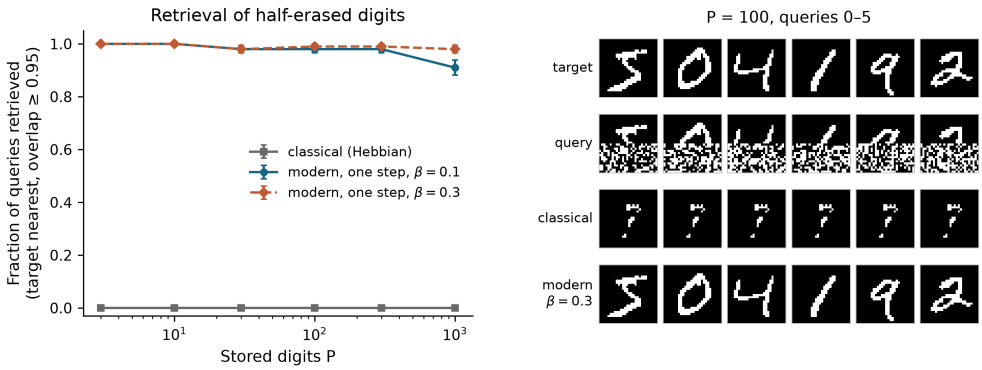
For $\beta \leq 0.01$ every query returns the same blurred average digit. At $\beta = 0.03$ the queried digit appears inside that average. For $\beta \geq 0.1$ one update returns the stored digit. Classical recall returns the same sparse fragment of strokes for every query.



(3) **Modern retrieval survives loads where classical recall fails.**

> Plot retrieval success against P for classical and modern recall. Add a gallery of targets, queries and both retrievals at P=100.

Modern retrieval at $\beta = 0.3$ succeeds for 98 to 100% of queries at every P up to 1000. At $\beta = 0.1$ success falls to 91% at $P = 1000$. The few failures are queries whose visible half matches a near-duplicate stored digit at least as well. Classical recall succeeds for no query at any P, even $P = 3$, where the load is 0.004. Correlation between digits, not load, defeats it. Your modern success at $\beta = 0.3$ should be at least 0.90 for every P, and your classical success should be zero.



Final verification

Use the figures to explain the difference between an unknown target distribution, an empirical distribution, a fitted model, and a sampling algorithm. Show where CD approximates a likelihood gradient, and how clamping turns generation into inference of missing pixels. Explain why the unstable-bit fraction follows the crosstalk curve and where recall of random patterns fails. Show that the modern Hopfield update and softmax attention compute the same vector, and how β changes the retrieved image.

Wrap up the work

```
Update README.md with each evidence figure beside its producing command. Explain the
reconstruction/generation comparison, the storage capacity and the attention form of
modern Hopfield retrieval. Retain dataset provenance and record training settings and
environment setup. Push the source, environment and evidence.
```

Show the completed evidence at the start of Week 7. Nothing is submitted. The final project is defended in Week 13.